

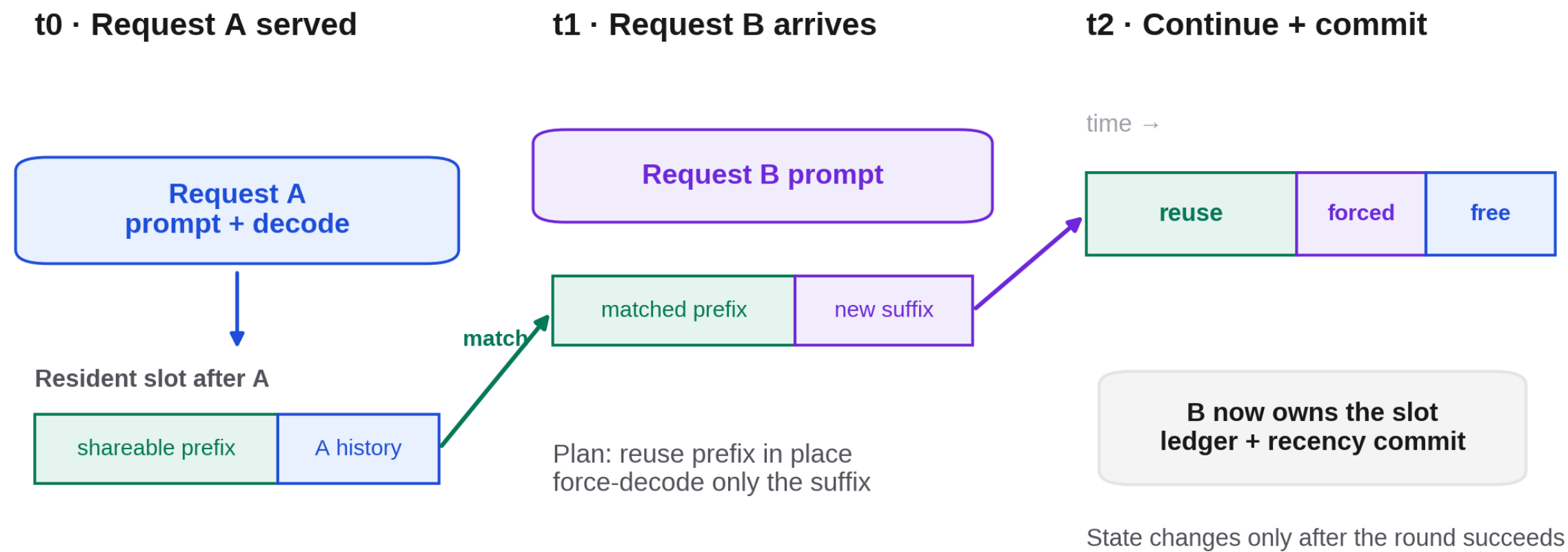
WEEKLY COLLABORATOR DISCUSSION

In-PE KV Reuse

Measured benefits, execution constraints, and an on-wafer secondary KV tier.

MODEL	EVIDENCE	SCOPE	DATE
Qwen3-1.7B	Real CS-3 TSC	M1 reuse + M3 exploration	2026-08-11

A later request can reuse KV left by an earlier request



Example: B shares A's resident system prefix; the appliance skips that work, rebuilds only B's suffix, then decodes normally.

One slot across three time steps: match the resident prefix, force only the new suffix, then continue normal decode and commit the new resident history.

CURRENT FEATURE SURFACE

Resident KV is now a serving resource, not a one-request scratchpad

ROUND BEHAVIOR

Reuse, rebuild, or replace

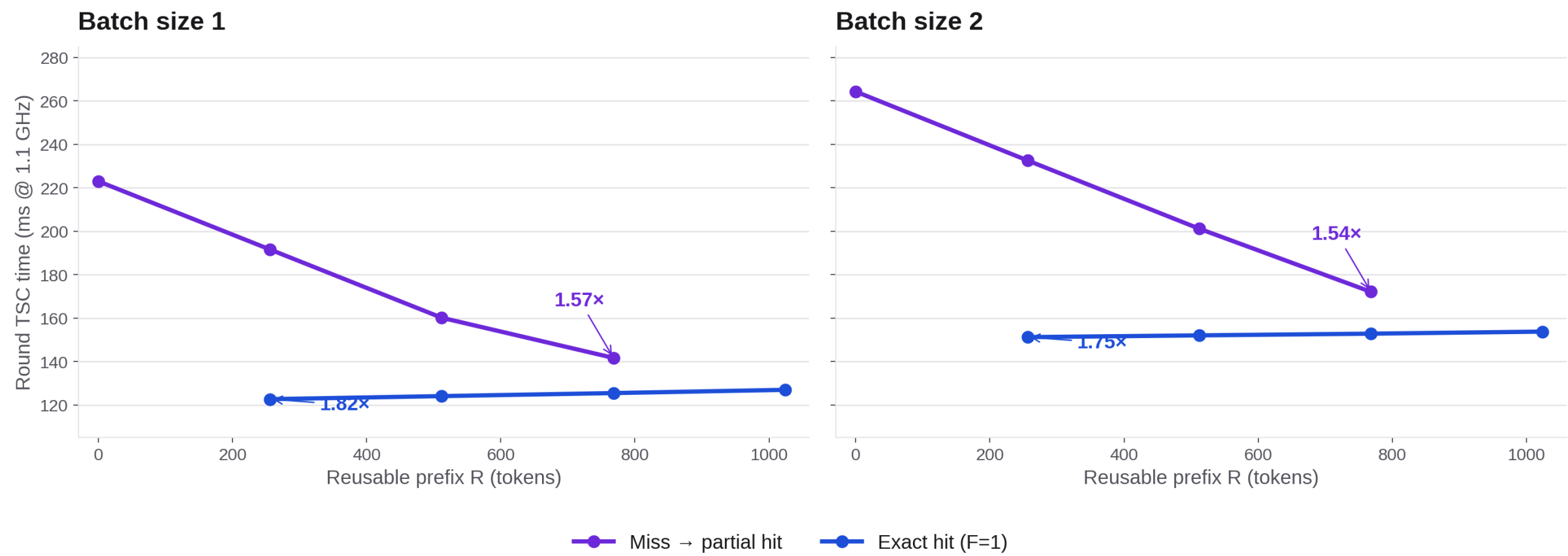
The host chooses a one-to-one resident slot assignment for the batch. The appliance starts from a shared safe prefix, receives no host KV payload on this path, and extends the slot with real prompt and sampled tokens.

ENABLED SCENARIOS

- 01** Hit — reuse in place; exact retains one seed, prefix forces only prompt[start:]
- 02** Miss — start at zero and rebuild the prompt by force-decode
- 03** Pressure — use an empty slot, otherwise replace one whole slot by LRU

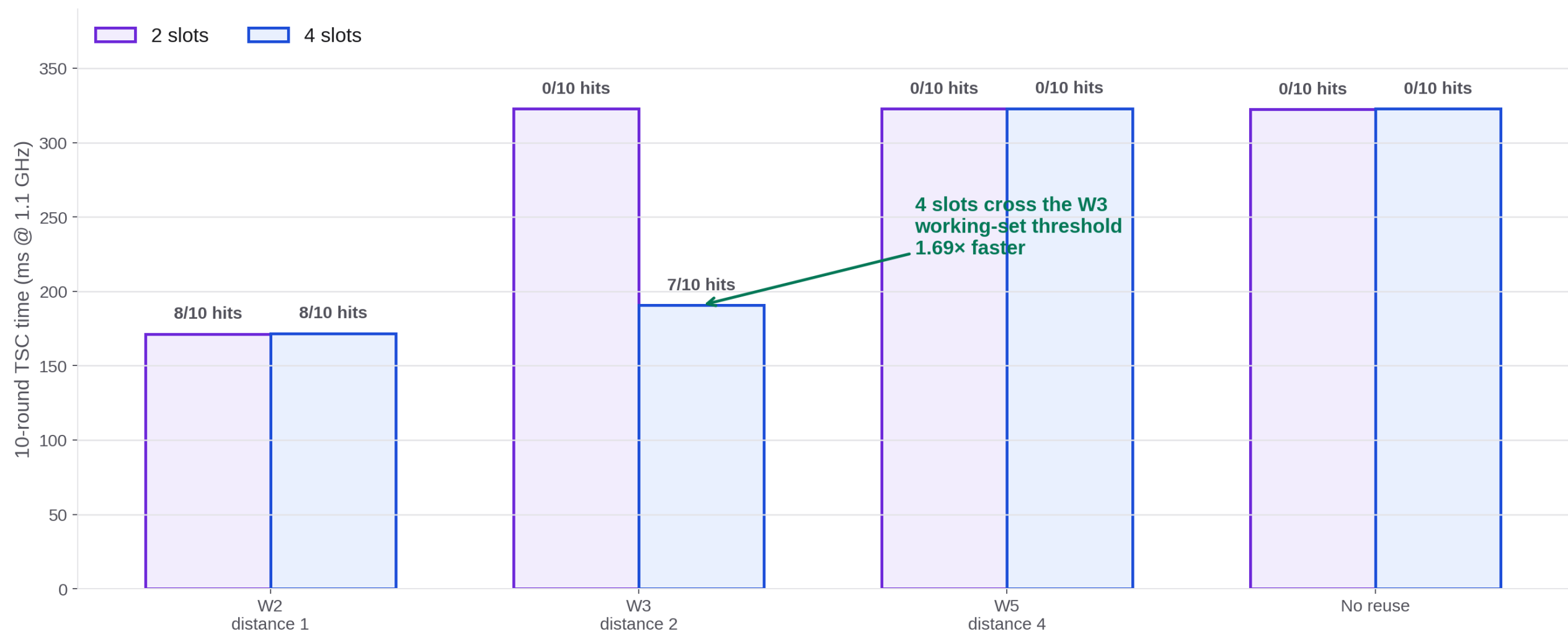
More reusable prefix translates into less round work

Longer resident prefixes reduce total round work



Full Qwen3-1.7B, real WSE-3, three repetitions per point. bsz 3–4 reached the PE-memory compile boundary (no TSC); partial shortens the suffix, while exact holds F=1.

More slots help only when they cross the working-set threshold



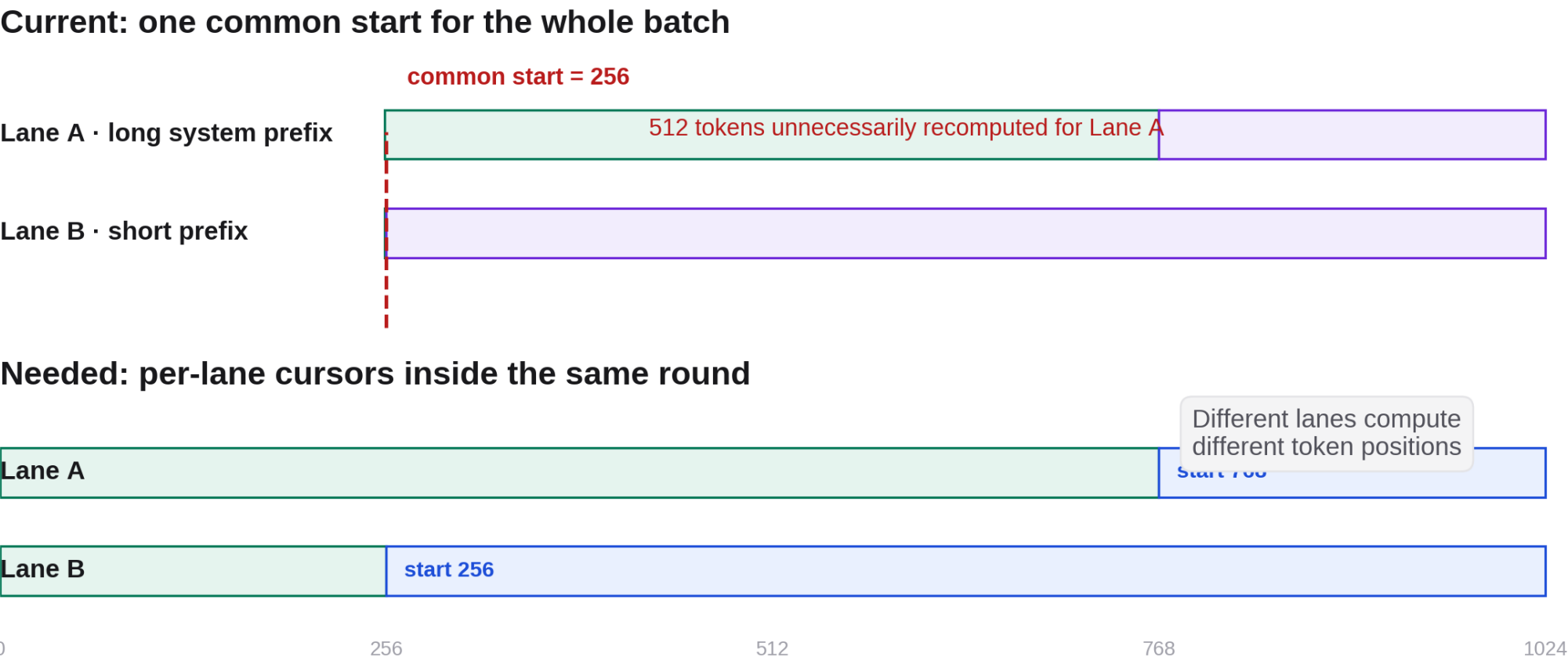
Ten-round synthetic traces, fresh sequence IDs, 256-token shared prefixes. W3 thrashes with two slots but produces seven content hits with four slots: 1.69× lower TSC than no reuse.

LIMITATION

Matching works; divergent execution is still missing.

Today every lane in a round is forced onto one common safe start, even when its reusable prefix is longer.

One shared start is unfair to requests with long reusable prefixes



Example: Lane A can reuse 768 tokens and Lane B only 256. The current batch starts both at 256; ragged execution would let the same PE block compute different token positions per lane.

Seed and decode KV can produce different local reuse depths

P = 8 example: host seed is contiguous; later decode appends round-robin

PE row				
0	seed 0	seed 1	dec 16	local depth 3
1	seed 2	seed 3	dec 17	local depth 3
2	seed 4	seed 5	dec 18	local depth 3
3	seed 6	seed 7	dec 19	local depth 3
4	seed 8	seed 9		local depth 2
5	seed 10	seed 11		local depth 2
6	seed 12	seed 13		local depth 2
7	seed 14	seed 15		local depth 2

**A 20-token logical prefix
lands at different local depths**

Current mitigation
round down to one safe boundary
(drop < one P-token block)

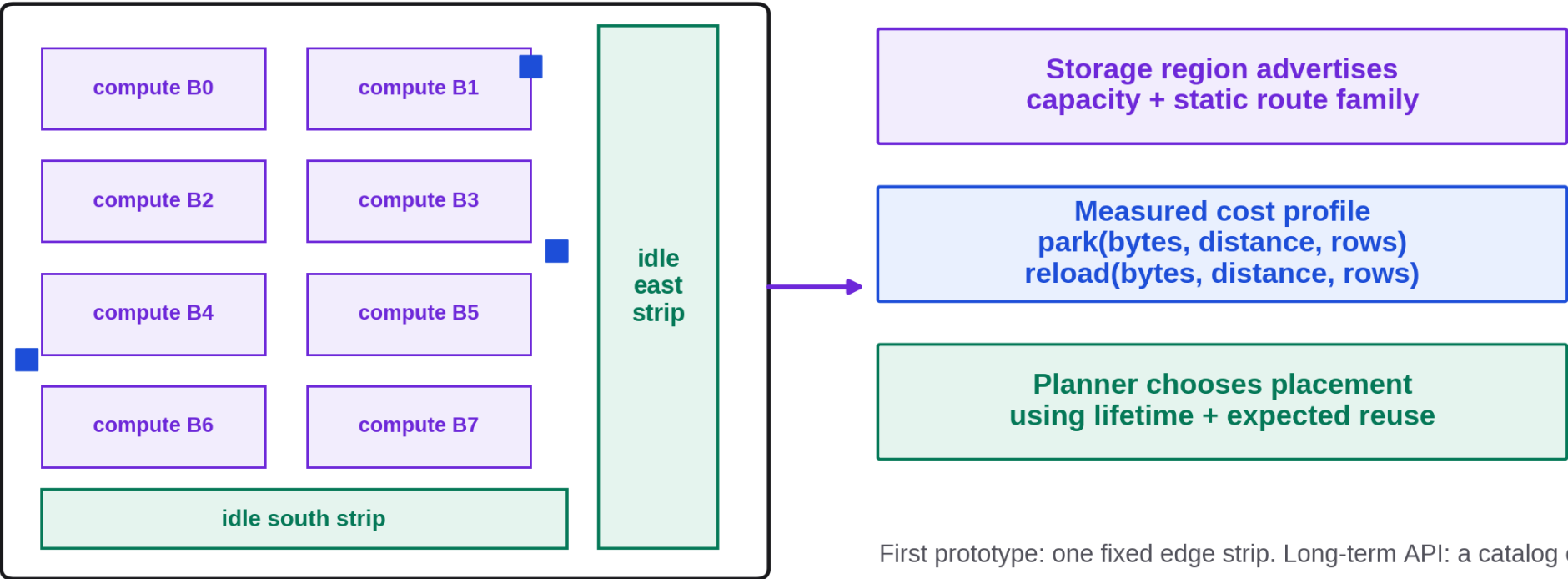
Full solution
per-row reuse cursors + addressing
may move control on-chip

Priority: lower for now
Block edge is modest; truncation cost is bounded, but must be measured.

After a contiguous host seed and round-robin decode appends, one logical prefix can end at different local indices across PE rows. The current safe mitigation truncates to a shared boundary.

Support flexible placement through measured, static movement profiles

Direction: placement-agnostic policy, statically realizable movement



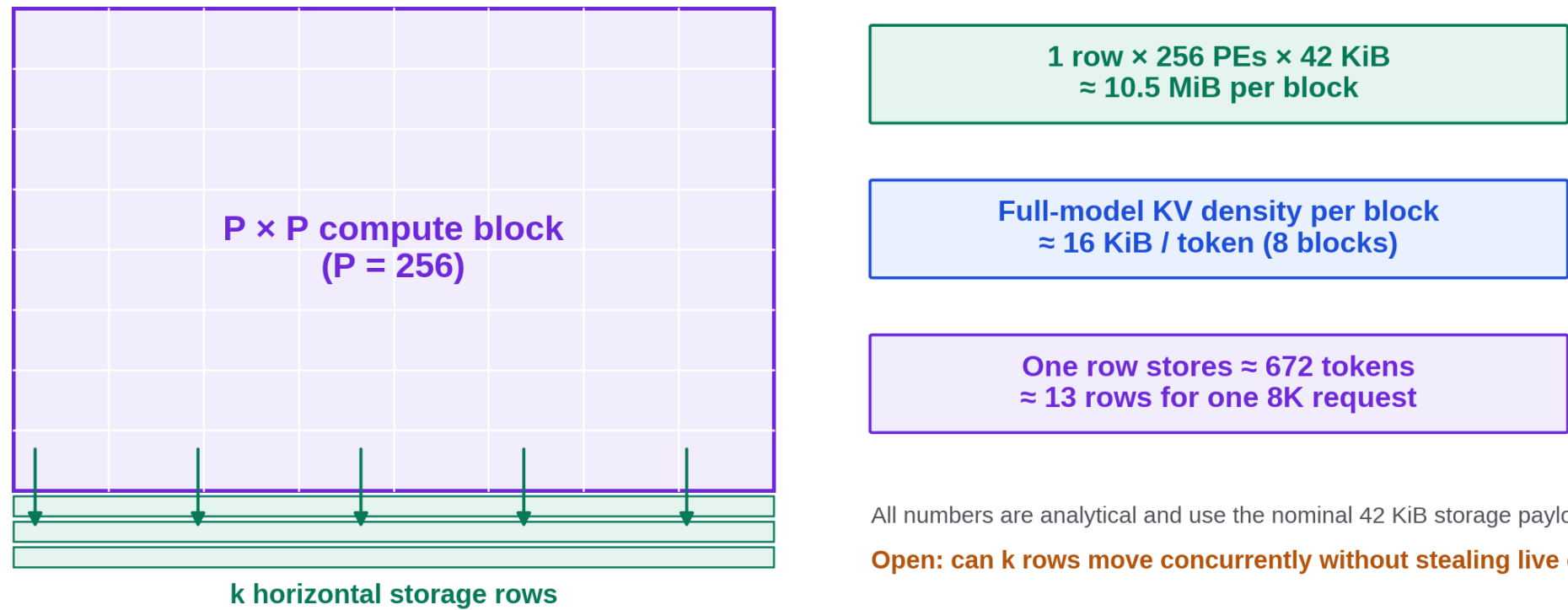
Candidate storage can live in any usable region — but routes are not keyed at runtime.

First prototype: one fixed edge strip. Long-term API: a catalog of precompiled/static placements.

Logical placement can be general, but WSE routes are not keyed by request ID. Each usable storage region should expose capacity plus measured park/reload curves for a precompiled route family.

A horizontal storage strip gives a simple first mechanism—and a clear capacity bill

Concrete probe: attach horizontal storage rows under each PE block

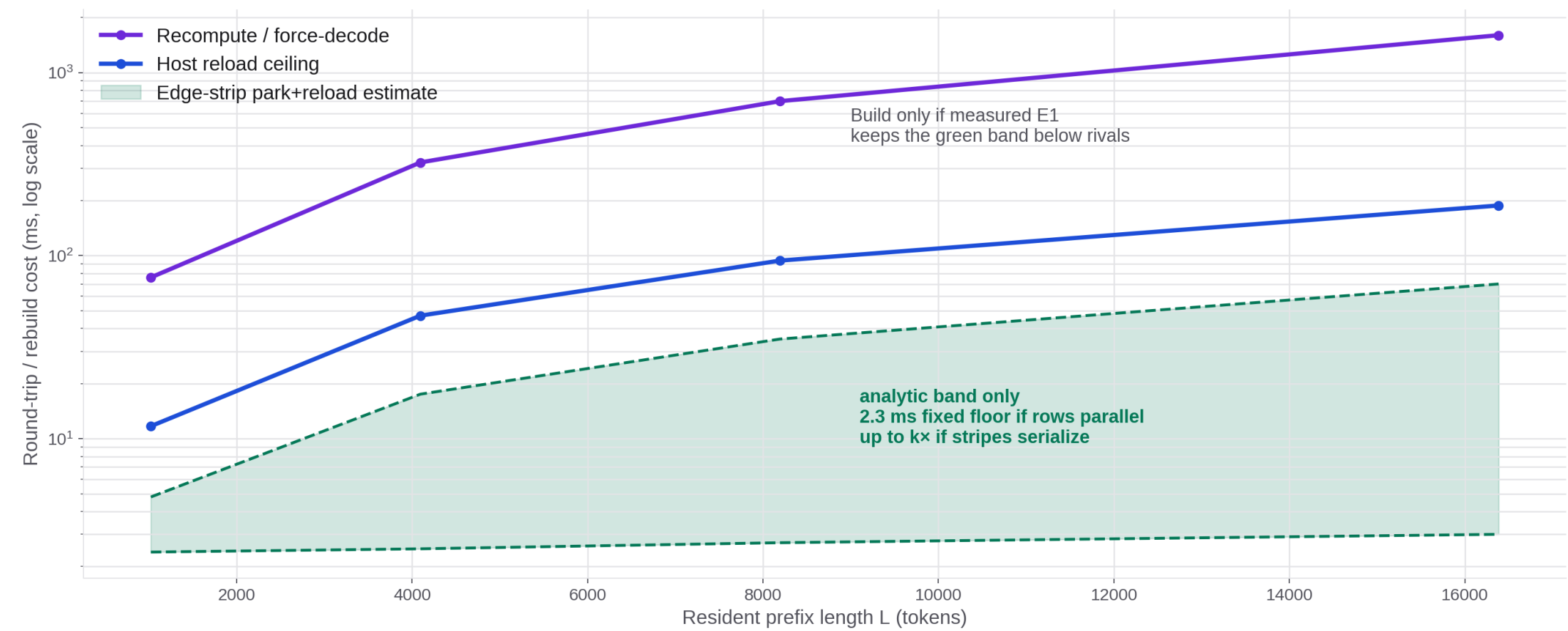


All numbers are analytical and use the nominal 42 KiB storage payload; E2 must compile-probe the real cap.

Open: can k rows move concurrently without stealing live colors/queues from decode?

Nominal analytical sizing: 42 KiB payload per storage PE and 128 KiB/token across the eight-block model. One 1×256 strip stores about 672 tokens per block; an 8K request needs about 13 rows.

Eviction is attractive only if the movement mechanism lands in the green band



Analytical edge-strip range: $2 \times P \times 4.54 \mu s \approx 2.3 \text{ ms}$ fixed round-trip if storage rows move in parallel; up to $k \times$ if rows serialize. E1 must measure payload, distance, row scaling and both directions.

What should we resolve next?

Ragged	Prioritize per-lane starts; choose the long-prefix success workload.
Row layout	Measure bounded reuse loss before moving per-row addressing on-chip.
M3-E1	Sweep park/reload by payload, distance, active rows and direction.
Placement + policy	Prototype one profiled edge strip; let measured cost drive eviction.